

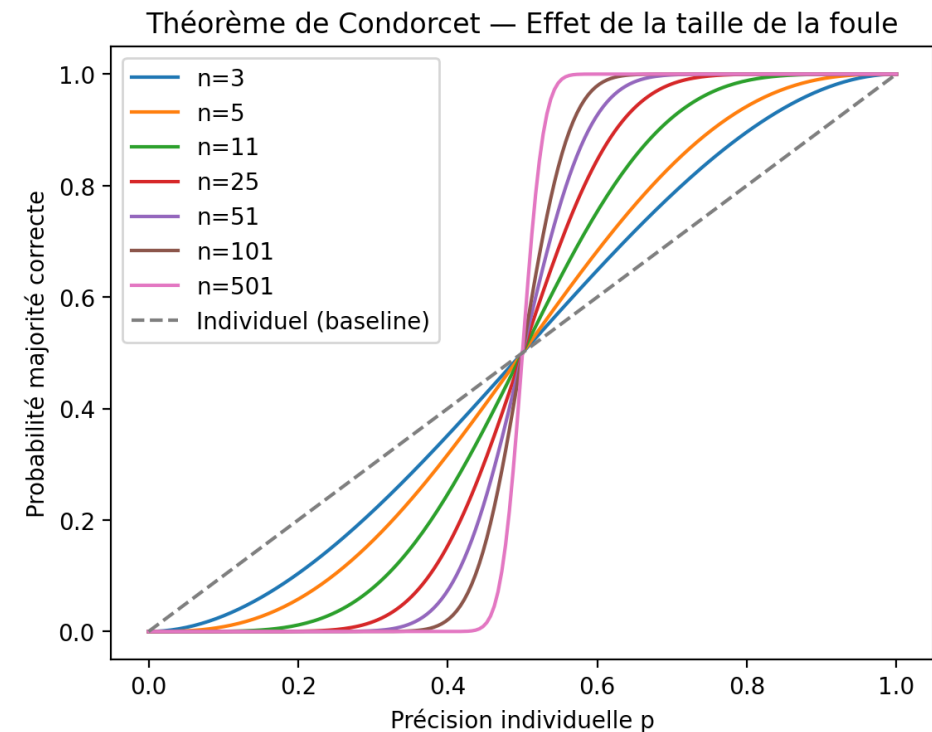
Lecture 4: Random Forests

Alexis BOGROFF

Course on Supervised Learning Problems @Albert School

January 28, 2026

- **Core Idea:** A large group of average predictors is smarter than a single expert.
- **Condorcet's Jury Theorem:** *"If it is more probable that each of the voters will judge in conformity with the truth, the more the number of voters increases, the greater the probability of the truth of the decision will be."*



Even slightly better-than-random classifiers achieve near-perfect accuracy when combined in large numbers.

The Theorem relies on two explicit conditions:

1. **Competence** ($p > 0.5$): Each voter (tree) must have a probability of being correct greater than random guessing.
2. **Independence**: Voters must be independent; their errors must be **uncorrelated**.
3. **Shared goal**.

The Mathematical Result:

- As the number of voters $N \rightarrow \infty$, the probability that the **majority vote** is correct approaches **1**.

Implication: We need many trees (N) that are **diverse** (Independent) and **decent** ($p > 0.5$).

- **Weak Learner:**
 - A model that performs slightly better than random guessing.
 - Example: A shallow Decision Tree (Stump), or a tree with high variance.
- **Strong Learner:**
 - A model that can achieve arbitrarily high accuracy.
 - Example: A Random Forest (an ensemble of weak learners).
- **Goal of Ensemble Methods:** Combine multiple weak learners to build a strong learner.

- **Bagging (Bootstrap Aggregating):**
 - Parallel training. Independent models. Reduces Variance.
 - *Example: Random Forest.*
- **Boosting:**
 - Sequential training. Models learn from previous mistakes. Reduces Bias.
 - *Example: AdaBoost, XGBoost (Lecture 6).*
- **Stacking:**
 - Meta-learning. A "final" model learns how to combine predictions from different base models (e.g., Linear Reg + KNN + Tree).

- **Intuition:** Errors cancel each other out. If one tree overestimates and another underestimates, the average is closer to the truth.

- **The Math:**

- If we have B independent random variables (trees) each with variance σ^2 :

$$\text{Var}(\text{Mean}) = \frac{\sigma^2}{B}$$

- **Impact:** As B increases, variance shrinks to 0 (if trees are independent).

- **The Reality (Trees are correlated):**

$$\text{Var}(\text{Forest}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- Because trees share training data, they are correlated ($\rho > 0$).
- **Random Forest Strategy:** By using random features, we lower ρ (correlation), maximizing the variance reduction.

- Random Forest is an ensemble learning method.
- It builds multiple decision trees and combines their outputs.
- Used for both classification and regression tasks.
- Helps reduce overfitting compared to a single decision tree.
- Model from Breiman's original paper (2001)

1. Bootstrapping (Sampling with Replacement)

- Generates multiple datasets of the same size as the original.
- Each sample contains about **63.2%** of unique data points.

2. Training Multiple Decision Trees

- Each tree is trained independently on a different bootstrap sample.

3. Trees use a Random Set of Feature at Each Split

- Instead of using all features, each split considers a **random subset**.
- **Classification:** Uses $m = \sqrt{d}$ features.
- **Regression:** Uses $m = d/3$ features.

4. Aggregation of Predictions

- **Classification:** Uses **majority voting** among trees.
- **Regression:** Averages predictions from all trees.

Bagging stands for **B**ootstrap **A**ggregating.

- **Objective:** Reduce variance and avoid overfitting.
- **The Process:**
 1. Given a dataset $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ of size n .
 2. Create B new datasets $D^1, D^2, \dots, D^B$ (also of size n).
 3. **Sampling with Replacement:** An observation can appear multiple times in the same sample D^b .
- **Result:** We obtain B slightly different training sets, leading to B diverse decision trees.

Question: If we sample n times with replacement from n items, what is the probability that a specific item x_i is *never* picked?

- Probability of NOT picking x_i in one draw: $1 - \frac{1}{n}$
- Probability of NOT picking x_i in n draws (independent):

$$P(\text{Not Selected}) = \left(1 - \frac{1}{n}\right)^n$$

- As $n \rightarrow \infty$, this limit converges to $1/e$:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = e^{-1} \approx 0.368$$

- **Conclusion:**
 - $\approx 36.8\%$ of data is left out (Out-of-Bag).
 - $\approx 63.2\%$ of data is used for training the tree.

What do we do with the 36.8% left out?

- For each tree, the unused samples are called **Out-of-Bag (OOB)** samples.
- They act as a **Natural Validation Set**. No need for Cross-Validation!

Computing the OOB Score:

1. For each instance x_i in the original dataset, identify all trees that **did not** see x_i during training.
2. Aggregate the predictions of these specific trees (Majority Vote or Average).
3. Compare this aggregated prediction to the true label y_i to calculate the error.

Note: In Scikit-Learn, set `oob_score=True` to compute this automatically.

Aggregating

- This aggregation is the mechanism to compute the overall prediction of the model by combining the independent predictions of each tree.
- It reduces variance over a simple Tree model by averaging multiple independent models (same than financial portfolio risk minimisation).
- **Classification:** Majority Voting.

$$\hat{y} = \arg \max_k \sum_{b=1}^B 1(y_b = k)$$

- **Regression:** Averaging.

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B y_b$$

- Since we use Bootstrapping, approx. **37%** of instances are left out for each tree.
- These are called **Out-of-Bag (OOB)** samples.
- **OOB Score Mechanism:**
 1. For every instance x_i , predict its label using *only* the trees that did NOT see x_i during training.
 2. Aggregate these predictions to get the OOB prediction.
 3. Compute accuracy/error on these predictions.
- **Advantage:** It acts as a free validation set! No need to do Cross-Validation (saving time).

Splitting Criteria:

- Reminder of metrics:
 - For Classification: **Gini Impurity** $G(X) = 1 - \sum_{i=1}^C p_i^2$
 - For Regression: **Mean Squared Error** $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$
- Random Feature Selection:
 - Each **node** considers only a random subset of m features.
 - This reduces the correlation between trees, improving generalization.
 - For Classification: $m = \sqrt{d}$
 - For Regression: $m = \frac{d}{3}$

- **Problem with simple Bagging:**
 - If there is one very strong feature in the dataset, **all** trees will likely choose it for the first split.
 - Result: All trees look similar → Predictions are correlated → Variance is not reduced effectively.
- **Random Forest Solution (Feature Randomness):**
 - At each split, we force the tree to choose from a random subset of features.
 - This forces trees to consider other features, creating diversity.
 - **Decorrelated trees** = Better aggregate performance.

- **Intuition:** If a feature is useful, it will be selected often for splits, and these splits will significantly reduce impurity (Gini or MSE).
- **Step 1: Single Tree Importance** For a single tree T , the importance of feature j is the sum of impurity decreases weighted by the number of samples at that node:

$$I_j(T) = \sum_{t \in T, v(t)=j} \frac{N_t}{N} \cdot \Delta \text{Impurity}(t)$$

- **Step 2: Forest Importance (The "Mean" part)** In a Random Forest with B trees, we average this score:

$$\text{Importance}_j = \frac{1}{B} \sum_{b=1}^B I_j(T_b)$$

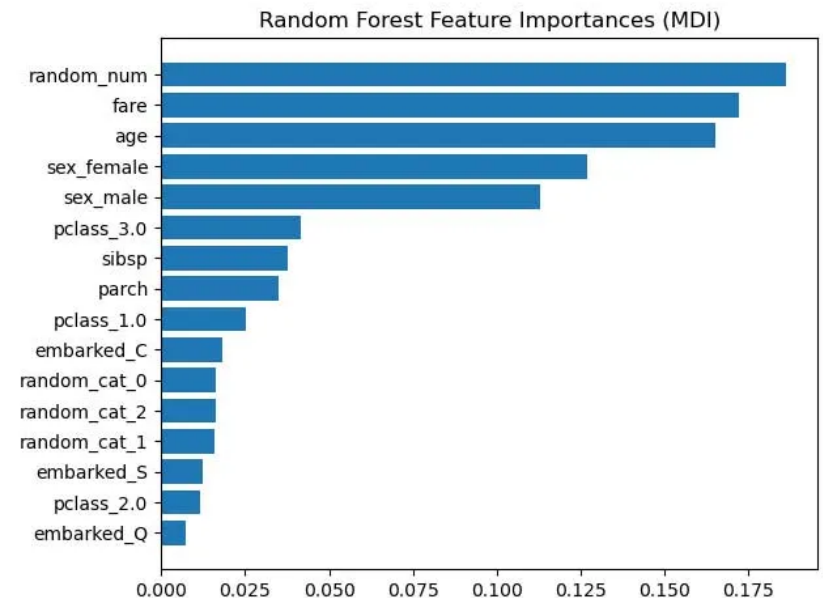
- **Normalization:** Scores are typically normalized so they sum to 1 (or 100).

Interpretation:

- Features with high scores "drive" the prediction.
- Useful for **Feature Selection** (dropping useless variables).

Warning: The Bias of MDI

- MDI is biased towards **High Cardinality Features** (numerical features with many unique values or categorical features with many categories).
- *Reason:* These features offer many potential split points, are selected more often by pure chance.
- **Alternative:** Use **Permutation Importance** (model-agnostic) to verify results if high cardinality features are present.



1. **Setup:** Choose number of trees B (e.g., 100) and feature subset size m .
2. **Training Loop:** For $b = 1$ to B :
 - Draw a bootstrap sample D^b of size N from the training data.
 - Grow a Decision Tree T_b on D^b .
 - At each node split:
 - ▶ Select m features at random from the total d features.
 - ▶ Pick the best split among these m features (Gini/MSE).
 - ▶ Split the node and recurse until a stopping criterion is met (e.g., min leaf size).
3. **Prediction:**
 - Pass the new instance x down all B trees.
 - **Regression:** Average the outputs: $\hat{y} = \frac{1}{B} \sum \hat{y}_b$.
 - **Classification:** Majority Vote.

- `n_estimators`:
 - Number of trees in the forest.
 - **Rule:** More is better (reduces variance), but slower. Value stabilizes after a point (e.g., 100 or 500).
- `max_features`:
 - Size of the random subset of features at each split.
 - Defaults: $\sqrt{d}$ (classification) or d (regression). Smaller = more diversity but more bias.
- `max_depth` / `min_samples_leaf`:
 - Controls the complexity of individual trees (Pre-pruning).

- Random Forest is "embarrassingly parallel".
- Since trees are independent, can be built simultaneously on different CPU cores.
- **In Scikit-Learn:**
 - Set `n_jobs=-1` to use all available cores.
 - Drastically reduces training time compared to Boosting (which is sequential).

- **Key Hyperparameters:**

- `n_estimators`: Number of trees (start with 100, then increase).
- `max_features`:
 - ▶ `"sqrt"`: Standard for Classification ($m = \sqrt{d}$).
 - ▶ `"log2"`: Alternative for very high-dimensional data ($m = \log_2(d)$).
- `n_jobs`: Set to `-1` to use all CPU cores (Parallelization).

Python Example

```
from sklearn.ensemble import RandomForestClassifier

# Initialize with parallel processing
rf = RandomForestClassifier(
    n_estimators=100,
    max_features='sqrt',    # or 'log2'
    max_depth=None,        # Fully grown trees
    n_jobs=-1,             # Use all cores
    random_state=42
)

rf.fit(X_train, y_train)
print(rf.feature_importances_)
```

Conditions for Optimal Performance:

- The data is **structured (tabular)**.
- There are **many features**, which may have weak individual predictive power.
- There are **complex feature interactions**, but deep learning would be overkill.
- The data is of **small to medium size**.
- The data is **noisy** (RF handles noise well via bagging).

Best Suited for Small to Medium-Sized Datasets:

- **Scalability and Training Time:** Handles moderately large datasets, but computational and memory demands increase significantly with size.
- **Interpretability:** Ideal for smaller datasets where feature importance scores provide actionable insights.
- **Data Requirements:** Performs well with limited data by combining weak signals into strong predictions, unlike deep learning which requires large datasets.

Caution:

- For very large datasets, consider alternatives like **Gradient Boosted Trees (e.g., XGBoost, LightGBM)** or **Deep Learning**.

Challenges in Business Adoption:

- **Interpretability Requirements:** Businesses prefer models like Logistic Regression that provide clear decision rules.
- **Computational Costs:** Random Forest is slow for large datasets, making XGBoost or LightGBM better alternatives.
- **Deployment Complexity:** Requires more memory and processing power compared to simpler models.
- **Regulatory Concerns:** Industries like finance and healthcare need transparent models to comply with regulations.
- **Preference for Simplicity:** Businesses often prioritize models that are easy to maintain and explain to stakeholders.

RF is commonly used in:

- **Exploratory Analysis:** Identifying important features before using simpler models.
- **Research and Prototyping:** Initial modeling before transitioning to more interpretable models for production.
- **Backend Systems:** Fraud detection, risk assessment, and anomaly detection where explanations are less critical.

Takeaway: RF is powerful but often replaced by simpler or more efficient models in production settings.

Key Comparison:

Aspect	Logistic Regression	KNN	Random Forest
Data Type	Num/Categ	Num/Categ	Num/Categ
Interpretability	High	Low	Medium
Overfitting Risk	Low if Regularized	Depends on K	Low
Computational Cost	Low	High (Inference)	Medium (Training)
Handles Missing Data	No	No	Yes
Non-Linearity	No	Yes	Yes
Large Datasets	Good	Bad	Average
Feature Importance	Yes	No	Yes